

Engineering Build Plan — Phase 0 to Phase 5

Step-by-step implementation roadmap: project setup, frontend, backend + API + database (PostgreSQL, Redis, Vector DB), AI orchestration, Intent/STT (on-device + cloud hybrid), and Maps integration.

Based on: Requirements & User Stories and Product & Technical Documentation v1.0 · Bharat Super App

Yeh document engineering ke liye ek practical, phase-wise build plan hai — exactly waise hi structure mein jaisa aapne bataya: Phase 0 mein project setup, Phase 1 mein poora frontend, Phase 2 mein backend ko frontend se connect karna (APIs + DB + Redis + Vector DB), Phase 3 mein AI orchestration, Phase 4 mein Intent / STT (on-device + cloud hybrid), aur Phase 5 mein Maps. Har phase mein: scope, kya exactly banana hai, tech choices (PDF documentation ke recommended stack ke hisaab se), aur deliverables diye gaye hain — taaki team isko seedha tracker mein todh sake.

Contents

1. Phase 0 — Project setup & foundations
2. Phase 1 — Frontend (poora, sabhi screens + design system)
3. Phase 2 — Backend connect with frontend: API gateway, Postgres, Redis, Vector DB
4. Phase 3 — AI orchestration layer (intent routing, RAG, reason & rank)
5. Phase 4 — Intent / STT — on-device + cloud hybrid
6. Phase 5 — Maps integration (MapmyIndia / Google Maps)
7. Cross-phase notes: sequencing, risk, and definition of done

PHASE 0

Project Setup & Foundations

Repo, tooling, design tokens, environments — sab kuch shuru karne se pehle

Scope

Phase 0 ka kaam hai poori team ke liye ek consistent, repeatable foundation banana — taaki Phase 1 (frontend) se directly code likhna shuru ho sake bina rukaavat ke.

Kya karna hai (step-by-step)

- **Monorepo / repo structure:** `/apps/mobile` (React Native ya PWA client), `/services/*` (Node.js/NestJS ya Go microservices), `/packages/ui` (shared design-system components) — Documentation ke Technical Architecture diagram (Client → Gateway → AI layer → Services → Data) ke layers ko hi folder structure mein map karo.
- **Tooling:** Git + branching strategy (trunk-based ya GitFlow), ESLint/Prettier, Husky pre-commit hooks, CI pipeline (GitHub Actions) for lint + test + build.
- **Environments:** dev / staging / prod — separate `.env` files, separate DB instances, separate API keys.
- **Design tokens setup:** design system ke colours ko CSS custom properties / Tailwind config mein convert karo — Deep Saffron #FF7A00 (primary), Royal Blue #0057FF (secondary), Emerald #22C55E (success), Alert Red #FF3B30 (emergency), Ink #121212 / Off-white #FAFAF8 (light/dark surfaces). Poppins font load karo (Google Fonts ya bundled), Roboto Mono data ke liye.
- **Theming scaffolding:** `--bg`, `--card`, `--text` jaise CSS variables define karo taaki light/dark dono themes runtime par toggle ho sakein (PRD requirement).
- **Project management:** tracker (Jira/Linear) mein E1–E10 epics aur unki stories (E1-S1, E2-S3, etc.) import karo — Story IDs already stable hain traceability ke liye.
- **Decide hosting/infra:** cloud provider (AWS/GCP/Azure), containerisation (Docker), and a basic CI/CD deploy target for staging.

Deliverables

- Empty-but-running repo with CI green
- Design tokens + theme switch working on a blank screen
- Tracker populated with all epics/stories from the Requirements doc

Duration guide: ~3–5 din (1 sprint ka chhota hissa), ek baar ke baad reuse hoga poore project mein.

PHASE 1

Frontend — Poora Build

Sabhi 8 screens + design system + navigation, AI engine se bina connect kiye

Scope

Is phase mein pura UI banta hai — React Native (ya Flutter) mein, jo Documentation ke "Recommended stack" ke mutabik hai ("One codebase, true iOS+Android native"). Iss stage par AI backend abhi connect nahi hota — mock/static data se UI test hota hai.

Information architecture (5 bottom-nav tabs)

Tab	Contains	Key screens
Home	AI search, quick actions, recents, local alerts	Home hub, AI Chat
Explore	Area intelligence & local services	Search area, Area Score, categories, property, nearby
Travel	Fuel, CNG, EV, parking, toilets, road trips	Map + categories, station list, road-trip planner
Government	Schemes, eligibility, documents	AI prompt, eligibility checker, scheme results
Profile	Saved items & settings	Profile, saved, settings

Health, Electricity/Utilities aur Emergency top-level tabs nahi hain — woh Home se quick actions aur AI deep-links se access hote hain (nav ko 5 tabs tak limited rakhne ke liye).

Step-by-step build order

- **Step 1.1 — Shared component library:** buttons, cards, chips, bottom-nav, rich-card shells (pharmacy card, scheme card, fuel card, area-score card), CTA button variants (directions / apply / view / remind / call). 2px-stroke line icons, no emoji in chrome.
- **Step 1.2 — Onboarding & auth screens (E1):** 3-slide onboarding + language picker, phone+OTP screen with resend timer, biometric unlock fallback flow, location-permission rationale screen (deny ho to bhi app manual area-entry ke saath chalta rahe — koi dead end nahi).
- **Step 1.3 — Home (E3):** large AI search bar (rotating placeholders), 8 quick-action tiles (Medicines, Fuel, Area, Schemes, Electricity, Explore, Water, Emergency), recents list (max 5), local alerts cards.
- **Step 1.4 — AI Chat shell (E2 UI only):** empty state (glowing blue+saffron orb + 4 suggestion chips), chat bubbles (user right/blue, AI left + optional rich card), typing-dot indicator, text+voice input bar — sab static/mock responses ke saath.
- **Step 1.5 — Explore / Area Intelligence (E4):** area-score circle UI, category grid (Safety, Health, Schools, Internet, Water&Power, Traffic), property stats block, nearby counts, save/compare UI.
- **Step 1.6 — Health (E5):** medicine search → pharmacy list cards, generic-alternative + dosage block with disclaimer, prescription-scanner upload UI, reminder set UI.
- **Step 1.7 — Government schemes (E6):** category browse chips, eligibility-checker form (age/occupation/income/state/gender), scheme result cards with Apply CTA.
- **Step 1.8 — Travel/Fuel (E7), Emergency (E8):** map placeholder + category chips, crowd-colour-coded station list; emergency big-button screen (works without auth — build it as a fully separate, no-login-required route).

- **Step 1.9 — Profile & settings (E9):** saved items grouped by type, theme/language toggle, AI personality selector, privacy/data controls.
- **Step 1.10 — States & accessibility pass:** empty / loading / error / offline states for every screen; WCAG AA contrast, dynamic type, screen-reader labels, ≥ 44 px tap targets, reduced-motion respect for animations (fadeUp, bubbleIn, orb pulse, typing-dot — all 200–400ms).

Deliverables

- All 8 screens (light + dark theme) clickable end-to-end with mock data
- Shared component library documented (Storybook ya equivalent)
- PWA-installable build running identically on iOS/Android browser shells

PHASE 2

Backend Connect with Frontend

API gateway + domain services + PostgreSQL + Redis + Vector DB

Scope

Yahaan se real data flow shuru hota hai — frontend ke mock data ko asli API calls se replace karte hain. Architecture: Client → API Gateway → Domain Services → Data layer (jaisa Technical Architecture diagram mein hai).

Step 2.1 — API Gateway

- Node.js/NestJS (ya Go) mein stateless gateway banao: auth, rate-limiting, session handling.
- Session token har endpoint ke liye required, except `/auth/*` aur `/emergency/*` (yeh bina-auth kaam karne chahiye).

Step 2.2 — Core endpoints (indicative API surface)

Endpoint	Method	Purpose
<code>/auth/otp/request</code>	POST	Send OTP to phone
<code>/auth/otp/verify</code>	POST	Verify OTP, create session
<code>/area/score</code>	GET	Area score, categories, property, nearby
<code>/health/medicine</code>	GET	Pharmacies with stock & price
<code>/health/prescription</code>	POST	OCR a prescription image
<code>/schemes/eligibility</code>	POST	Match schemes to a profile
<code>/travel/stations</code>	GET	Fuel/EV/parking near a point
<code>/travel/roadtrip</code>	POST	Plan stops along a route
<code>/emergency/sos</code>	POST	Broadcast location to 112 + contacts
<code>/alerts</code>	GET	Active local alerts by area
<code>/profile</code>	GET/PATCH	Read/update profile, saved, settings

(`/ai/query` aur `/ai/feedback` Phase 3 mein banenge, jab AI layer ready ho.)

Step 2.3 — Database layer

- **PostgreSQL** — relational core entities: User, Conversation, Message, Area, Pharmacy, Scheme, FuelStation, Alert (jaisa Data Model mein defined hai). Schema migrations set up karo (Prisma/TypeORM ya equivalent).
- **Redis** — session cache, OTP rate-limiting counters, recents cache, hot-path data (e.g. frequently-requested area scores) cache karne ke liye.
- **Vector DB** — abhi Phase 2 mein sirf infra set up karo (e.g. pgvector ya standalone Pinecone/Weaviate); actual embeddings/RAG ka kaam Phase 3 (AI orchestration) mein hoga.

Step 2.4 — Domain microservices (stub → real)

- Area, Health, Govt, Travel, Utility, Emergency, Profile — har ek ek chhota independent service/module banao jo apne DB tables aur (jahan applicable) external integration (Maps API, Pharmacy stock, Scheme DB, Utility boards) ko call kare.
- Pehle stub/static JSON return karo per service, phir ek-ek karke real data source connect karo — taaki frontend integration parallel chal sake.

Step 2.5 — Frontend ↔ backend wiring

- Phase 1 ke mock data calls ko real API client (Axios/fetch wrapper + React Query/SWR) se replace karo.
- Loading/error/empty/offline states (jo Phase 1 mein UI-only design hui thi) ab real network responses ke saath test karo.
- OTP sign-in, biometric fallback, location-permission flow — end-to-end real session ke saath verify karo.

Deliverables

- Auth (OTP) end-to-end working
- Home, Area, Health, Govt, Travel, Emergency screens real data se populate ho rahe hain
- Postgres + Redis + Vector-DB infra provisioned aur services se connected

PHASE 3

AI Orchestration

Intent routing → context assembly → RAG → reason/rank → response builder

Scope

Yeh AI layer hi product ka centerpiece hai. Goal: `/ai/query` endpoint banana jo streaming text + cards + actions return kare, typed response contract ke saath: { text, cards[], actions[], sources[], confidence }.

Query lifecycle jo build karna hai

- **3.1 Intent classification:** query ko domain mein route karo — area / health / government / fuel-travel / utilities / emergency / general chat. Off-domain query par graceful capability message do.
- **3.2 Context assembly:** location, profile, saved items, aur chat history attach karo request ke saath.
- **3.3 Data retrieval (RAG):** Vector DB (Phase 2 mein provision hua) ko ab embeddings se populate karo — Scheme DB, Pharmacy stock, Area data ko semantic search ke liye index karo, taaki answers grounded rahe ("never fabricate" requirement).
- **3.4 Reason & rank:** LLM call jo concise, friendly answer synthesize kare aur options ko rank kare — AI personality "smart, helpful, friendly, trustworthy" tone follow kare, aur har answer ka basis cite kare (e.g. "Baner scores higher overall").
- **3.5 Rich result cards:** domain ke hisaab se card type return karo — Area Score card, pharmacy list, scheme card, fuel list, comparison answer.
- **3.6 Action layer:** CTA mapping — directions / apply / view / remind / call — deep-link context ke saath jo frontend module ko pre-load kare.
- **3.7 Feedback loop:** `/ai/feedback` endpoint —  ko query context ke saath store karo, future personalization ke liye.
- **3.8 Low-confidence handling:** jab confidence score kam ho, guess karne ki bajaye clarifying question return karo.

Tech notes (Documentation ke recommended stack se)

- AI orchestration = LLM + RAG over domain data — "grounded, current answers" ke liye.
- Streaming response zaroori hai taaki typing-indicator <1.2s aur full card <3s ka performance target meet ho.
- Data sources stale/down ho to gracefully degrade karo — kabhi fabricate mat karo (medical dosage aur scheme eligibility ke liye especially critical).

Deliverables

- `/ai/query` (streaming) aur `/ai/feedback` live
- Sabhi 6 domains ke liye intent-routing accuracy tested
- AI Chat screen (Phase 1 ka UI) ab real streamed answers + rich cards dikhata hai

PHASE 4

Intent / STT — On-device + Cloud Hybrid

Voice input, multilingual speech-to-text, hybrid latency strategy

Scope

Yeh phase voice-first experience ko power karta hai — "Ask by voice" (E2-S2) story ke liye. Documentation explicitly recommend karta hai: **on-device + cloud hybrid** STT — latency + multilingual coverage dono ke liye.

Kyun hybrid?

- **On-device STT:** fast, low-latency, offline-tolerant, low data-footprint (low-bandwidth mode requirement) — short/common phrases aur basic English/Hindi ke liye theek.
- **Cloud STT:** better accuracy regional languages/accents ke liye, especially jab network available ho — final transcription ko refine/correct karne ke liye use hota hai.

Step-by-step

- **4.1 Mic capture + live transcription UI:** mic tap karte hi on-device STT se live, editable transcription dikhao (E2-S2 acceptance criteria).
- **4.2 On-device model integration:** platform-native STT (iOS Speech framework / Android SpeechRecognizer ya on-device Whisper-tiny jaisa model) — Hindi + English support.
- **4.3 Cloud STT fallback/refinement:** jab network available ho, cloud STT (e.g. cloud speech API) se transcription ko refine karo, especially regional-language input ke liye.
- **4.4 Language detection:** Hindi vs English (vs future regional languages) auto-detect karo, taaki user ko language manually select na karna pade.
- **4.5 Intent router se connect:** final (edited/confirmed) transcript ko typed-query ki tarah hi `/ai/query` mein route karo — same intent-classification pipeline (Phase 3) reuse karo.
- **4.6 Edge-cases:** low-signal / offline graceful fallback (on-device-only mode), ambiguous transcription par clarifying prompt, noisy-environment handling.

Roadmap note

Launch scope = English + Hindi. 6+ regional languages roadmap par hain — string-externalisation aur STT model selection isi Phase 4 architecture mein extensible rakho taaki baad mein naye languages plug-in ho sakein (especially Farmer suite ke liye, jo fully voice-first hona zaroori hai).

Deliverables

- Voice input working end-to-end (mic → live transcript → edit → send → AI answer)
- Hindi + English accuracy benchmarked
- Hybrid fallback (on-device ↔ cloud) tested under poor network

PHASE 5

Maps Integration

MapmyIndia / Google Maps — geocoding, directions, nearby POIs

Scope

Maps almost har domain ko touch karta hai — Area Intelligence, Health (pharmacy directions), Travel/Fuel, Emergency (nearby hospitals). Documentation recommend karta hai **MapmyIndia / Google Maps** — "India-grade map coverage" ke liye.

Step-by-step

- **5.1 SDK/API setup:** Maps SDK (MapmyIndia ya Google Maps) ko client mein integrate karo, API keys env-wise (dev/staging/prod) secure karo.
- **5.2 Geocoding service:** address/area name ↔ lat-long conversion — Area Intelligence ke "search an area" flow ke liye.
- **5.3 Map preview component:** Area Score screen ke liye chhota static/interactive map preview banao (Phase 1 mein placeholder tha, ab real map render karo).
- **5.4 Nearby POI search:** hospitals, schools, police, ATMs, pharmacies, fuel stations, blood banks — Maps Places API se nearby counts aur lists fetch karo (Area, Health, Emergency, Travel modules ke liye reusable service banao).
- **5.5 Directions/navigation deep-link:** "Directions" CTA tap karne par OS-native maps app (Google Maps / Apple Maps) ko navigation intent ke saath open karo.
- **5.6 Route planning (road-trip):** origin→destination route ke along fuel stops, food, EV chargers, stays, attractions plot karo (Travel/Fuel module, E7-S2).
- **5.7 Live crowd/availability overlay:** fuel-station crowd-level (■■■■) aur EV-charger/parking live-availability data ko map markers par color-code karo.
- **5.8 Low-bandwidth handling:** lazy-load maps, image-light tiles — "low-bandwidth mode" non-functional requirement ke mutabik.

Deliverables

- Maps SDK live across Area, Health, Travel, Emergency screens
- Directions deep-link working on iOS + Android
- Road-trip planner returning real route + stops

Cross-phase notes

Sequencing logic

Yeh phase order isliye chosen hai: Phase 1 (frontend) standalone ban sakta hai mock data ke saath, isliye usse pehle shuru karke parallel-track mein Phase 2 (backend infra) bhi start ho sakta hai — dono teams alag-alag kaam kar sakte hain aur Phase 2 ke end mein wire-up karte hain. Phase 3 (AI) Phase 2 ke data layer (especially Vector DB) par depend karta hai, isliye uske baad aata hai. Phase 4 (voice) Phase 3 ke intent-router ko reuse karta hai. Phase 5 (Maps) kisi bhi point par parallel ho sakta hai kyunki woh ek largely independent integration hai — lekin usko Phase 2 ke baad rakha gaya hai taaki nearby-POI services ready ho.

Phasing vs. product roadmap (P1–P4)

Yeh 0–5 engineering phases Documentation ke product-level **P1** ("Real AI engine + Health, Area, Govt live data — First production release") ko deliver karte hain. P2 (Travel, Utilities, Emergency hardened + Hindi), P3 (Farmer suite) aur P4 (Payments, proactive AI, civic, jobs) is plan ke baad ke release cycles hain.

Definition of done (har phase ke har story ke liye)

- Acceptance criteria QA mein pass — iOS aur Android dono par
- Light + dark theme dono correct, sabhi phone sizes par layout sahi
- Empty / loading / error / offline states implemented
- Accessibility checks pass (labels, contrast, targets, dynamic type)
- Analytics events firing key actions ke liye
- Strings externalised (no hard-coded copy) — localization-ready
- Telemetry/logging + error handling in place, no console errors
- Code reviewed, merged, automated tests covered